Backtests and the Code Behind the Top 5

PyQuantLab Following 8 min read · 1 hour ago

Want to explore and test the strategies behind this dashboard? The **Mega Backtrader Strategy Pack** includes **500+ Backtrader-ready Python strategies**, batch runners, interactive dashboards, CSV metrics, chart exports, and documentation. You can get the complete package here: Mega Backtrader Strategy Pack.

Mega Backtrader Strategy Pack - 500+ Backtrader Strategies | PyQuantLab

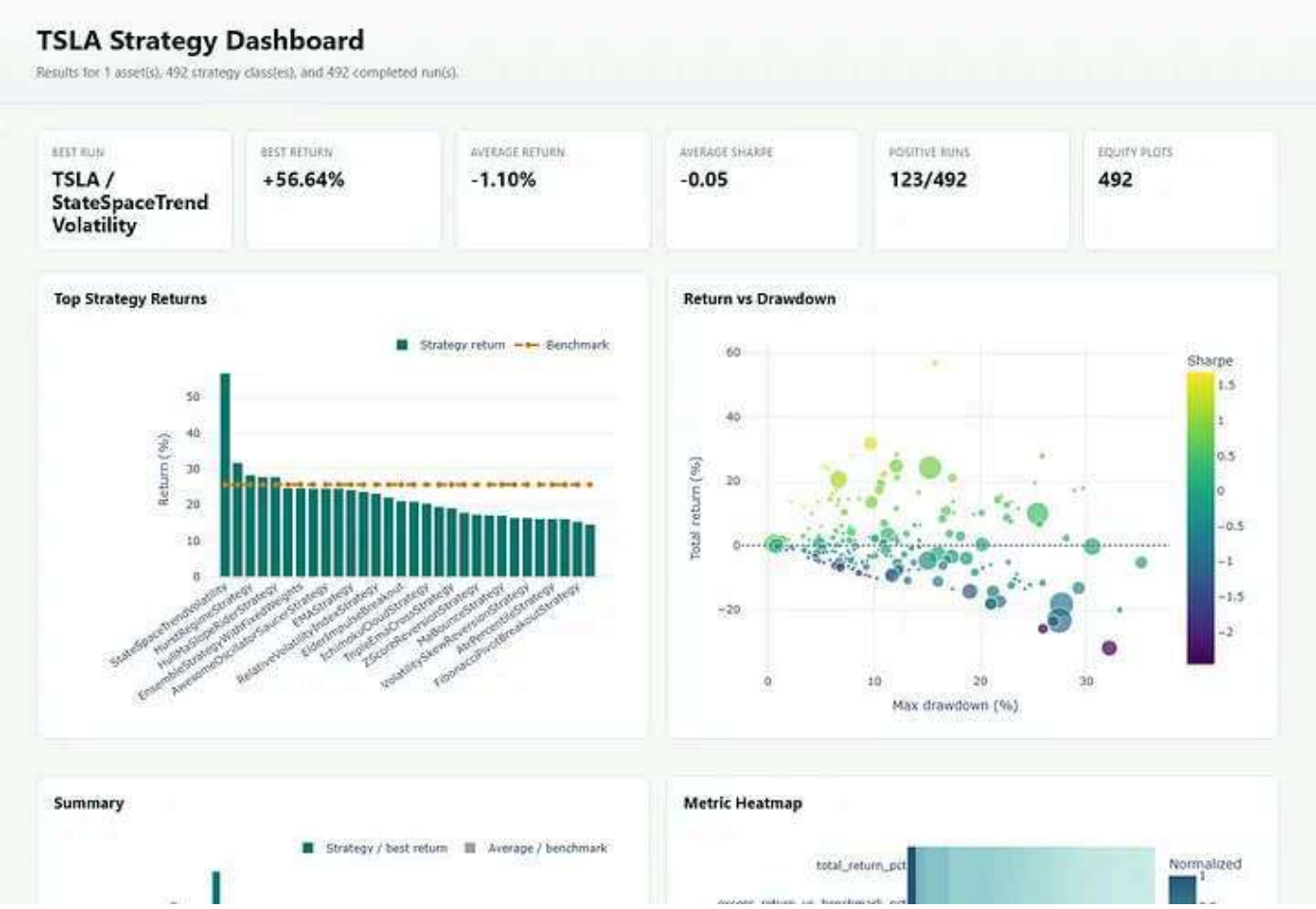
Skip months of development. Get 500+ Backtrader-ready Python strategies with TA-Lib indicators, fast batch backtests...

www.pyquantlab.com

Testing one trading strategy can answer a narrow question. Testing hundreds of strategies under the same assumptions gives you something more useful: context.

The **TSLA Strategy Dashboard** compares 492 completed Backtrader runs over a one-year daily-data window from **June 20, 2025 through June 18, 2026.** Every strategy started with $10,000. The dashboard ranks returns, compares risk and reward, displays normalized metrics, and provides an equity chart for every completed run.

Open the interactive TSLA Strategy Dashboard.



What the Dashboard Says at a Glance

The headline figures immediately show why broad testing matters:

Dashboard statistic	Result
Strategy classes tested	492
Completed runs	492
Positive-return runs	123
Positive-return rate	25.00%
Average strategy return	-1.10%
Average Sharpe ratio	-0.05
Best strategy return	56.64%
SPY benchmark return	25.65%
TSLA buy-and-hold return	24.31%

Only one quarter of the strategies finished with a positive return. The average result was negative even though TSLA and SPY both rose by roughly 25%. This is the first important lesson from the dashboard: a large strategy library is a research universe, not a list of automatic winners.

The top-return chart also provides a clean cutoff. Only the first five ranked strategies beat the dashboard's **25.65% SPY benchmark**. The sixth-ranked strategy returned 24.71%, so the top five are the strategies that produced positive excess return against SPY in this snapshot.

The Top Five TSLA Strategies

Rank	Strategy	Return	Excess vs SPY	Sharpe	Max drawdown	Closed / open trades
1	`StateSpaceTrendVolatility`	56.64%	30.99%	1.49	15.76%	Feb-00
2	`AdaptiveVWAPMeanReversionStrategy`	31.70%	6.04%	1.56	9.67%	Oct-00
3	`HurstRegimeStrategy`	28.37%	2.71%	1.26	12.12%	Feb-00
4	`BasicVolatilityMomentum`	27.83%	2.18%	0.82	25.85%	1-Jan
5	`HullMaSlopeRiderStrategy`	27.81%	2.15%	1.68	7.93%	Jan-00

All five beat both SPY and TSLA buy-and-hold during the test, but the margins after the first strategy were modest. Four of the five also completed two trades or fewer. Those small samples make the rankings interesting research leads, not strong evidence of repeatability.

1. State Space Trend Volatility

Dashboard result: 56.64% return, 1.49 Sharpe ratio, 15.76% maximum drawdown, and two closed trades.

Source: `StateSpaceTrendVolatility.py`

The dashboard winner uses a statistical state-space model instead of a conventional moving-average crossover. Every 30 bars, it takes the latest seven closes, converts them to logarithms, and fits a local linear trend with `statsmodels`:

```
model = sm.tsa.UnobservedComponents(
    log_ts_data,
    level="lltrend",
)
```

```
result = model.fit(
    method="lbfgs",
    disp=False,
    maxiter=500,
)
```

The strategy stores an estimated trend level and slope. A sufficiently positive slope opens a long position; a sufficiently negative slope opens a short

position. If the signal changes direction while a position is open, the strategy closes first and considers reversing on the following bar:

```
if self.trend_slope > self.p.trend_slope_threshold_buy:
    if current_position_size == 0:
        self.order = self.buy()
    elif current_position_size < 0:
        self.order = self.close()
```

```
elif self.trend_slope < self.p.trend_slope_threshold_sell:
    if current_position_size == 0:
        self.order = self.sell()
    elif current_position_size > 0:
        self.order = self.close()
```

The appeal is clear: state-space models try to separate an underlying trend from noisy observations. The result also deserves the most skepticism. A 56.64% return from only two closed trades can be dominated by one well-timed position.

There is also an implementation detail worth reviewing before further research. `statsmodels` normally exposes `filtered_state` with states on one axis and time on the other. The code currently reads:

```
self.trend_level = filtered_state[-1, 0]
self.trend_slope = filtered_state[-1, 1]
```

That indexing should be verified against the installed `statsmodels` version. If the intended values are the latest level and slope states, the time-axis

orientation matters. A corrected implementation would be a different strategy and should receive a fresh backtest.

2. Adaptive VWAP Mean Reversion

Dashboard result: 31.70% return, 1.56 Sharpe ratio, 9.67% maximum drawdown, and ten closed trades.

Source: AdaptiveVWAPMeanReversionStrategy.py

This is the most extensively traded strategy in the top five and the only one with ten completed trades. It calculates a rolling 20-day volume-weighted average price, measures the standard deviation of price around VWAP, and looks for stretched prices in weak-trend conditions.

The long setup requires price below the lower VWAP boundary, ADX below 30, and RSI below 45. The short setup mirrors those conditions:

```
weak_trend = adx < self.params.adx_weak_threshold
```

```
price_oversold = price < lower_2std
rsi_oversold = rsi < self.params.rsi_oversold

if price_oversold and weak_trend and rsi_oversold:
    self.entry_atr = atr
    self.order = self.buy()

price_overbought = price > upper_2std
rsi_overbought = rsi > self.params.rsi_overbought

if price_overbought and weak_trend and rsi_overbought:
    self.entry_atr = atr
    self.order = self.sell()
```

The primary target is VWAP: a long closes when price rises back to VWAP, while a short closes when price falls back to it. ADX above 35 also triggers an exit because a strengthening trend can be hostile to mean reversion.

Risk management adapts as price approaches VWAP. The strategy begins with a wider ATR-based trail and switches to a tighter trail inside half a standard deviation:

```
if self.using_tight_trail:
    trail_distance = self.params.atr_trail_mult_tight * atr
else:
    trail_distance = self.params.atr_trail_mult_wide * atr
```

Among the top five, this result has the most balanced combination of trade count, excess return, and drawdown. Ten trades are still far too few to establish robustness, but they provide more information than a one-trade or two-trade result.

3. Hurst Regime Strategy

Dashboard result: 28.37% return, 1.26 Sharpe ratio, 12.12% maximum drawdown, and two winning closed trades.

Source: `HurstRegimeStrategy.py`

`HurstRegimeStrategy` changes its behavior according to the estimated market regime.

- A Hurst exponent above 0.55 is treated as persistent or trend-following.

- A Hurst exponent below 0.45 is treated as mean-reverting.
- Values between those thresholds produce no new entry.

In a trending regime, MACD crossovers generate entries. In a mean-reverting regime, stochastic crossovers at extreme readings generate entries:

```
if current_hurst > self.p.hurst_trend_threshold:
    if self.macd_cross[0] > 0:
        self.order = self.buy()
    elif self.macd_cross[0] < 0:
        self.order = self.sell()
```

```
elif current_hurst < self.p.hurst_reversion_threshold:
    if self.stoch.percK[0] < self.p.stoch_oversold \
            and self.stoch_cross[0] > 0:
        self.order = self.buy()
    elif self.stoch.percK[0] > self.p.stoch_overbought \
            and self.stoch_cross[0] < 0:
        self.order = self.sell()
```

Once in a position, the strategy maintains a manual trailing stop three ATRs away from the best price reached since entry.

The 100% win rate looks attractive, but it represents only two completed trades. The more useful research question is whether regime switching remains beneficial across longer periods, other Hurst lookbacks, and assets with different volatility structures.

4. Basic Volatility Momentum

Dashboard result: 27.83% return, 0.82 Sharpe ratio, 25.85% maximum drawdown, one closed trade, and one position still open.

Source: SimplifiedRegimeMomentum.py

The class name is `BasicVolatilityMomentum`, and its logic is exactly what the name suggests. It divides 14-day price momentum by the 20-day standard deviation of daily returns:

```
self.momentum = bt.indicators.PctChange(
    self.data.close,
    period=self.params.momentum_window,
)
```

```
self.volatility = bt.indicators.StandardDeviation(
    bt.indicators.PctChange(self.data.close, period=1),
    period=self.params.vol_window,
)

signal = momentum / volatility
```

A positive signal above 0.02 opens a long position. A negative signal below -0.02 opens a short. An existing position closes only when the absolute signal falls below half the entry threshold:

```
if signal > self.params.signal_threshold and not self.position:
    self.buy()
elif signal < -self.params.signal_threshold and not self.position:
    self.sell()
elif abs(signal) < self.params.signal_threshold / 2 \
        and self.position:
    self.close()
```

This strategy nearly matched the other top-five returns, but its risk statistics were much weaker. Its 25.85% maximum drawdown was the largest of the group, and its 0.82 Sharpe ratio was the lowest. The final return also includes one position marked to market at the last bar, so it is not a fully realized result.

5. Hull MA Slope Rider

Dashboard result: 27.81% return, 1.68 Sharpe ratio, 7.93% maximum drawdown, and one winning closed trade.

Source: HullMaSlopeRiderStrategy.py

The Hull moving average is designed to reduce lag while remaining smooth. This strategy calculates a 20-period Hull MA, compares its current value with its value three bars ago, and filters direction with a 50-day SMA:

```
slope = self.hull[0] - self.hull[-self.p.slope_lookback]
up_trend = self.data.close[0] > self.sma[0]
dn_trend = self.data.close[0] < self.sma[0]
```

```
if not self.position:
    if slope > 0 and up_trend:
        self.order = self.buy()
    elif slope < 0 and dn_trend:
        self.order = self.sell()
```

The strategy exits when the Hull MA slope flips and also installs a Chandelier-style trailing order three ATRs away:

```
amt = self.atr[0] * self.p.stop_atr_mult
side = self.sell if is_long else self.buy
self.trail = side(
    exectype=bt.Order.StopTrail,
    trailamount=amt,
)
```

On the dashboard, this strategy produced the best Sharpe ratio and lowest maximum drawdown among the top five. However, both statistics came from a single completed trade. Its `notify_order` callback also places a new trailing order after every completed order, including an exit fill. Entry and exit fills should be distinguished explicitly before treating the result as a clean test of the documented rules.

What the Ranking Does — and Does Not — Tell Us

The dashboard is valuable because every strategy is visible under a shared evaluation framework. It makes several comparisons immediate:

- The winning state-space model generated the most return, but with only two trades.
- Adaptive VWAP had the strongest evidence within the top five because it completed ten trades and kept drawdown below 10%.
- Basic Volatility Momentum earned a similar return to the third- and fifth-ranked systems but did so with far more drawdown.
- Hull MA Slope Rider had the best risk-adjusted headline numbers, yet only one completed trade.
- Exactly five strategies beat SPY; most of the 492 runs failed even to produce a positive return.

What the dashboard cannot establish is future performance. These are in-sample, single-asset results from one one-year window. Ranking hundreds of strategies also creates selection risk: even if every strategy had no durable edge, a few could still look exceptional by chance.

A Better Research Workflow After the Dashboard

The dashboard should be the beginning of the investigation:

1. Inspect the source code and confirm that the implementation matches the strategy description.
2. Separate entry fills, exit fills, protective orders, and reversals in the order state.
3. Reject or down-weight results with tiny trade samples.
4. Retest promising strategies over longer and non-overlapping periods.
5. Run walk-forward or out-of-sample validation.
6. Test parameter sensitivity rather than trusting a single default configuration.
7. Include realistic commissions, slippage, borrowing constraints, and position sizing.
8. Compare against both the traded asset and an external benchmark.

That workflow turns a leaderboard into a research tool. The **Mega Backtrader Strategy Pack** provides the strategy modules, batch runner, metrics, charts, and dashboards needed to move through that process without rebuilding the infrastructure for every new idea.

Get the complete package here: Mega Backtrader Strategy Pack.

Disclaimer

This article is for research and educational use only. Backtest results are not financial advice, investment advice, or a guarantee of future performance. The dashboard is a historical snapshot, and the top-ranked strategies were selected after comparing hundreds of tests. Validate code behavior, data quality, sample size, costs, slippage, execution assumptions, and out-of-sample performance before using any trading system.

Trading + Cryptocurrency + Stock Market + Algorithmic Trading + Python +

Written by PyQuantLab

1K followers · 6 following

Quantitative finance with Python. Books, code, apps, and research tools.

Following